



Builder form

Give entity editors a shared structure and validation vocabulary.

Problem and boundary

Public contract

Working integration

@epam/ai-dial-builder-form



When this library is useful

Give entity editors a shared structure and validation vocabulary.

01

Build an editor

Compose metadata and setup sections in a consistent page.

02

Share field rules

Use one validator for name and version policies.

03

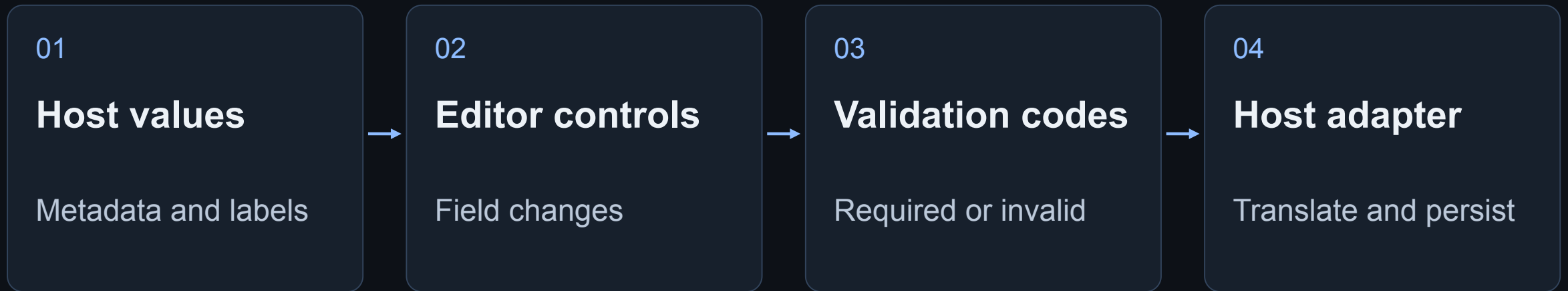
Localize metadata

Edit additional deployment names and descriptions by locale.



Follow the ownership boundary

Inputs and effects stay explicit.



Library: Editor shells, field controls and pure validation error codes.



The public contract to start with

EditorLayout · selected surface; full API index is in the bundle.

Input or operation	Contract
title	string
onBack	() => void
leftContent (optional)	ReactNode
rightContent (optional)	ReactNode



A minimal workspace integration

TypeScript / React example · imports and host setup abridged.

```
<EditorLayout
  title="Create demo deployment" onBack={onBack}
  leftContent={
    <EditorSection title="Metadata">Demo fields</EditorSection>
  }
  rightContent={
    <EditorSection title="Setup">Host setup</EditorSection>
  }
/>
```

Full example: [examples/builder-form.tsx](#) · uses [@epam/ai-dial-builder-form](#)



Wire a realistic host workflow

Keep builder form focused on its public responsibility.

01

Compose metadata

Place shared fields inside the editor sections.

02

Validate values

Map validation error codes to localized field messages.

03

Save through the host

Submit the validated values using the application adapter.



Validate without translating

The host maps error codes into messages.

```
const codes = validateDeploymentCreationFields(  
  { name: '', description: '', iconUrl: '',  
    version: '1.0.0', topics: [], otherLocales: [] },  
  { validateNamePattern: true, validateVersionPattern: true },  
);
```

Focused source-backed example · see [examples/](#) and [content/api-index.md](#).



Available customization

01

Layout composition

EditorLayout exposes leftContent, rightContent and actions.

02

Metadata fields

The shared creation form includes avatar, version, topics and locales.

03

Policy switches

Name and version pattern checks are opt-in validator options.



Find the integration and its checks

Existing tests are evidence of intended behavior; suite results are not claimed.

Evidence	Where to look
Real consumer	<code>prompt-editor/src/components/PromptEditor/PromptEditor.tsx</code>
Test evidence	renders all shared fields
Test evidence	renders the host file manager modal when open



Constraints that affect integration

01

Avatar ownership

Resolve `iconPreviewUrl` and open file selection in the host.

02

Locale visibility

The locale editor hides when no locale options are supplied.

03

State is mixed

Some controls manage transient state; persistence remains outside.